



Example

Core Idea: Sliding Window + Sorted Set

The pattern here is **Sliding Window** combined with an **Ordered Set** for efficient range queries.

The Key Insight

You need to check if *any two elements within a window of size* `indexDiff` have values within `valueDiff` of each other.

Instead of checking every pair ($O(k^2)$ per window), you maintain the window in **sorted order** so you can binary search for a value-neighbor in $O(\log k)$.

What the Sorted Set is Actually Doing

Think of it as a **sorted "notepad"** of the last `indexDiff` numbers you've seen.

For each new number, you ask: *"Is there anything already on this notepad that's within $\pm valueDiff$ of me?"*

```
nums = [1, 5, 9, 1, 5, 9], indexDiff=2, valueDiff=3
```

```
Process 1: notepad=[]          → add 1 → notepad=[1]
Process 5: notepad=[1]        → is anything in [2,8]? No → add
5 → notepad=[1,5]
Process 9: notepad=[1,5]      → is anything in [6,12]? No → add
9 → notepad=[1,5,9]
                                ^^window full, remove nums[0]=1
→ notepad=[5,9]
Process 1: notepad=[5,9]      → is anything in [-2,4]? No → add
1 → notepad=[1,5,9]
                                ^^window full, remove nums[1]=5
```

```
→ notepad=[1,9]
...

```

The Binary Search Step Explained

```
pos = window.bisect_left(num - valueDiff)
```

This jumps **directly to the smallest number $\geq$ (num - valueDiff)**.

```
num = 5, valueDiff = 3
Valid range = [5-3, 5+3] = [2, 8]

Sorted window = [1, 4, 7, 10]
                  ↑
bisect_left(2) lands here (index 1, value 4)

Is 4 within [2, 8]? YES → return True
```

You only need to check **one candidate** — the smallest value $\geq$ lower bound. Why?

- If even the closest value (from the left boundary) is out of range, nothing else will be in range either.

Pattern Recognition Checklist

Signal in the problem	Pattern to think of
<code>abs(i - j) <= k</code>	Sliding Window of size k
<code>abs(val[i] - val[j]) <= t</code>	Sorted structure for range query
Both constraints together	Sliding Window + Sorted Set

Complexity

	Complexity
Time	$O(n \log k)$ — n elements, $O(\log k)$ per insert/search

	Complexity
Space	$O(k)$ — window holds at most <code>indexDiff</code> elements

vs. brute force $O(n \cdot k)$ — the sorted set turns the value-range check from a linear scan into a binary search.

Full Trace: `nums = [1, 5, 9, 1, 5, 9]`, `indexDiff=2`, `valueDiff=3`

Step 1 — Process `nums[0] = 1`

```
notepad = []
range to search = [1-3, 1+3] = [-2, 4]
bisect_left(-2) → index 0, but notepad is EMPTY → no candidate
add 1 → notepad = [1]
window size = 1, indexDiff = 2 → no removal yet
```

Step 2 — Process `nums[1] = 5`

```
notepad = [1]
range to search = [5-3, 5+3] = [2, 8]
bisect_left(2) → lands at index 0, value = 1
Is 1 inside [2, 8]? NO (1 < 2) → no candidate
add 5 → notepad = [1, 5]
window size = 2, indexDiff = 2 → no removal yet
```

Step 3 — Process `nums[2] = 9`

```
notepad = [1, 5]
range to search = [9-3, 9+3] = [6, 12]
```

bisect_left(6) → lands at index 2, but index 2 is OUT OF BOUNDS → no candidate

add 9 → notepad = [1, 5, 9]
window size = 3 > indexDiff=2 → REMOVE nums[left] = nums[0] = 1
notepad = [5, 9], left moves to 1

Step 4 — Process `nums[3] = 1`

notepad = [5, 9]
range to search = [1-3, 1+3] = [-2, 4]
bisect_left(-2) → lands at index 0, value = 5

Is 5 inside [-2, 4]? NO (5 > 4) → no candidate

add 1 → notepad = [1, 5, 9]
window size = 3 > indexDiff=2 → REMOVE nums[left] = nums[1] = 5
notepad = [1, 9], left moves to 2

Step 5 — Process `nums[4] = 5`

notepad = [1, 9]
range to search = [5-3, 5+3] = [2, 8]
bisect_left(2) → lands at index 1, value = 9
[1, 9]
↑
smallest value >= 2 is 9

Is 9 inside [2, 8]? NO (9 > 8) → no candidate

add 5 → notepad = [1, 5, 9]
window size = 3 > indexDiff=2 → REMOVE nums[left] = nums[2] =

9

notepad = [1, 5], left moves to 3

Step 6 — Process `nums[5] = 9`

notepad = [1, 5]

range to search = [9-3, 9+3] = [6, 12]

bisect_left(6) → lands at index 2, OUT OF BOUNDS → no candidate

add 9 → notepad = [1, 5, 9]

window size = 3 > indexDiff=2 → REMOVE nums[left] = nums[3] = 1

notepad = [5, 9], left moves to 4

Result → `return False` ✓

Why bisect_left only checks ONE value

notepad = [1, 9], searching for num=5, range=[2,8]

[1, 9]

↑

bisect_left(2) lands here

If 9 is already > 8 (out of range HIGH side),
every value to the RIGHT would also be > 8.

Every value to the LEFT (just 1) is < 2, out of range LOW side.

So if this ONE landing spot fails → nothing in notepad can match.

That's why you never need to check more than one element — the sort order guarantees everything left is too small, everything right is too large.

Left Pointer

left doesn't point into the sorted set

left points into the **original nums array**, which never changes.

```
nums = [1, 5, 9, 1, 5, 9]  ← this never moves, never reorders
      ↑
      left moves along HERE, not inside the sorted set
```

They are two separate structures

```
nums = [1, 5, 9, 1, 5, 9]  ← original array, fixed order
      0  1  2  3  4  5
      ↑
      left=1 means "oldest element is nums[1] = 5"
```

```
notepad = [1, 5, 9]  ← sorted set, reordered, just for searching
```

left never indexes into **notepad**. It always does **nums[left]** to get the value to remove.

Concrete Example

```
left=1, notepad=[1, 5, 9]
```

we want to remove the oldest element

```
→ nums[left] = nums[1] = 5  ← go to ORIGINAL array
```

```
→ notepad.remove(5)
→ notepad = [1, 9]
→ left = 2
```

Even though `5` is at index 1 in the sorted set, it doesn't matter — you're telling the set **what value** to delete, not **which position**.

One-liner Summary

`left` tracks position in `nums` (fixed), not in the sorted set (reordered). You always ask *"what value was oldest?"* via `nums[left]`, then remove that value from the set.

Summary

The Problem asks two things simultaneously

```
1. abs(i - j) <= indexDiff    → indices must be close
2. abs(nums[i] - nums[j]) <= valueDiff → values must be close
```

Sorted Set (notepad)

Stores the last `indexDiff` numbers seen. Kept sorted for fast searching.

Window size cap (eviction)

Guarantees index constraint. Notepad never holds more than `indexDiff` elements, so anything inside it was seen recently enough.

The range `[num - valueDiff, num + valueDiff]`

Translates the value constraint into a range. Instead of checking `abs(a-b) <= valueDiff` for every pair, you just ask *"does anything exist between these two numbers?"*

`bisect_left(num - valueDiff)`

Jumps directly to the smallest value that could possibly satisfy the range. One lookup instead of scanning every element.

The single check after bisect

If the landed value is within the range → both constraints met → return True. If not, nothing else in the notepad can match either (everything left is too small, everything right is too large).

The split responsibility

Index constraint → enforced by eviction (notepad size)
Value constraint → checked by bisect + range

Why only check the LEFT boundary element

```
notepad = [1, 3, 7, 12]
range   = [2, 8]
```

```
bisect_left(2) lands on 3
                ↑
```

Everything LEFT of 3 → values are `< 2` → too small → impossible ❌

3 itself → is `3 <= 8`? → check this ONE value ✅

Everything RIGHT of 3 → values are `> 3` → if we needed them, we'd have found 3 first anyway

Key insight

You don't need the EXACT boundary, you just need to know if **anything exists inside the range**.

So you jump to the **first possible candidate** (left boundary). If even that fails, the range is empty — nothing inside it.

```
If 3 is inside [2,8] → FOUND, return True
If 3 is outside [2,8] meaning 3 > 8 → everything right is also > 8 → EMPTY range
```

One check is enough.

The eviction logic guarantees the number in the notepad is always $\leq \text{indexDiff}$

```
window.add(num)                # add current number

if len(window) > indexDiff:      # if window too big
    window.remove(nums[left])    # remove OLDEST number
    left += 1                   # slide left forward
```

Every single step, you **never let the notepad grow beyond `indexDiff` size**.

Concrete walkthrough — `indexDiff = 2`

```
index 0: add nums[0]=1 → notepad=[1]          size=1 ✓
index 1: add nums[1]=5 → notepad=[1,5]        size=2 ✓
index 2: add nums[2]=9 → notepad=[1,5,9]      size=3 ✗ TOO BIG

                remove nums[0]=1
                notepad=[5,9]                 size=2 ✓
```

index 3: add nums[3]=1 → notepad=[1,5,9] size=3 ❌ T00 B
IG

remove nums[1]=5
notepad=[1,9] size=2 ✅

index 4: add nums[4]=5 → notepad=[1,5,9] size=3 ❌ T00 B
IG

remove nums[2]=9
notepad=[1,5] size=2 ✅

The size limit IS the guarantee

At index 4, notepad has exactly 2 elements.
Those 2 elements can ONLY be nums[3] and nums[4].

Why? Because:

- you add one element per step
- you remove one element when size exceeds indexDiff
- so notepad always holds the LAST indexDiff elements

indexDiff=2 means:

notepad at index 4 = {nums[3], nums[4]}

↑
farthest back = index 4-2 = 2...
wait, nums[4] is current,
so farthest back is index 3 = 4-1

Actually notepad = last indexDiff PREVIOUS elements
= nums[3], nums[4] at this moment
(current not yet added when searching)

One-line summary

Notepad size is capped at `indexDiff` by forcibly evicting the oldest element every time it grows too big — so mathematically it can never contain anything older than `indexDiff` steps.
